第1批讲义（新版）：从 GPT 往回拆

自顶向下 · 按需回溯 · 阶段 0

怎么读这份讲义

旧版是“先打地基再盖楼”：导数 → micrograd → MLP → GPT。这一版倒过来：**先看楼，再一层层往下拆**，就像反向传播从 loss 往回走一样。

每一层先提一个问题，然后回答它。回答里用到的数学或工具，放在**回溯框**里：

> **回溯框**长这样：只讲这一层够用的那一点，配一个小数字例子和几行能跑的代码。完整推导不在这里，统一留给“深挖章节”。

整份讲义只围绕一个文件：`gpt.py`（一个 0.21M 参数的字符级 GPT，在莎士比亚文本上训练）。所有代码片段都已在 CPU 上跑过，输出是真实结果。

层	问题	回溯框里补的
0	它到底在做什么?	——
1	它怎么决定下一个字?	概率分布、softmax、采样
2	那些分数从哪来?	向量、矩阵乘法、张量形状
3	字怎么变成数字?	one-hot、embedding、位置编码
4	一个字怎么“看到”前面的字?	点积、加权平均
5	Q/K/V 和因果掩码是什么?	下三角矩阵、−∞、除以 $\sqrt{d}$
6	一个 Transformer 块里还有什么?	ReLU、残差、均值方差、LayerNorm
7	它怎么知道自己猜得好不好?	log、交叉熵
8	它怎么越学越好?	导数、梯度下降、链式法则（直觉）
9	训练为什么会出问题?	学习率、过拟合、初始化

第 0 层：它到底在做什么?

运行（环境不用你装，交给我也行）：

```
python gpt.py cpu 3000
```

训练前（第 0 步）让它写字，得到的是乱码：

```
k yacInKh cSgWFXMu
MnDIWA?.fMuZqLx
Lmx3 a$UorF&c,wPLmZepo$MtFUmsQ,':n3 b TyNTUy'
```

训练 3000 步（CPU 上几分钟）后：

```
BENVOLIO:
My bladier to tose there nor deaduned his,
Who, oncreatwer your woundy after, king too:
And me is not reptward's splityiend.

ISABELLA:
Sir, my nour cave prairps ploty.
```

不懂英语也能看出：它学会了剧本格式（人名 + 冒号 + 换行）、常见单词（my、your、king、Sir），以及“像英语”的拼法，虽然很多词是它编的。

整个 GPT 只做一件事：给定前面的字，猜下一个字。 生成一段文字 = 猜一个字 → 把它接到末尾 → 再猜下一个，循环 300 次。对应 `gpt.py` 里的 `generate`：

```python
for _ in range(max_new_tokens):
    logits, _ = self(idx[:, -block_size:])        # 看前面最多 64 个字
    probs = F.softmax(logits[:, -1, :], dim=-1)   # 变成下一个字的概率
    idx = torch.cat([idx, torch.multinomial(probs, 1)], dim=1)  # 抽一个，接上
```

下面所有层，都在回答这三行代码背后的问题。

第 1 层：它怎么决定下一个字?

字表里一共 65 个字符（大小写字母、标点、空格、换行）。模型每次输出 **65 个分数**，叫 `logits`，每个字符一个。分数越高，模型越觉得下一个是它。

但分数可以是负数，加起来也不等于 1，不能直接当概率用。所以要用 **softmax** 把它变成概率。

> **回溯框 1：概率分布与 softmax**
>
> **概率分布**：一组非负数，加起来等于 1。比如 “下一个字是 e 的概率 0.66，是 a 的概率 0.24，是 o 的概率 0.10”。
>
> **softmax 两步走**：
>
> 1. 每个分数取指数 e^x（e ≈ 2.718）。指数的好处：结果永远是正数，而且分数大的会被放得更大。
> 2. 每个结果除以它们的总和，让总和变成 1。
>
> 小例子，三个分数 [2.0, 1.0, 0.1]：
>
> - 第 1 步：e^2.0 = 7.389，e^1.0 = 2.718，e^0.1 = 1.105，总和 11.212

- 第 2 步：7.389 ÷ 11.212 = 0.659，2.718 ÷ 11.212 = 0.242，1.105 ÷ 11.212 = 0.099

```
logits = torch.tensor([2.0, 1.0, 0.1])
print(F.softmax(logits, dim=-1))   # tensor([0.6590, 0.2424, 0.0986])
```

采样：不总是选最大的，而是按概率“抽签”。抽 10 次大约 6–7 次是第 0 个，偶尔抽到别的，这样生成的文字才不会每次都一样。

```
torch.multinomial(torch.tensor([0.659, 0.242, 0.099]), 10, replacement=True)
# tensor([2, 1, 0, 2, 0, 1, 0, 0, 0, 0])  —— 0 出现了 6 次
```

本层结论：模型输出 65 个分数 → softmax 变成概率 → 按概率抽一个字。

第 2 层：那 65 个分数从哪来?

模型最后一行是 `self.lm_head = nn.Linear(n_embd, vocab_size)`。意思是：模型内部用 **64 个数**描述“当前这个位置的理解”，`lm_head` 把这 64 个数变成 65 个分数。这一步就是**矩阵乘法**。

回溯框 2：向量、矩阵乘法、张量形状

向量 = 一排数，比如 [1, 2]。在模型里，一个字在某个位置的“意思”就是一个向量（这里长度 64）。

矩阵 = 一张数字表格。**向量 × 矩阵**：用向量去和矩阵的每一列做“对应相乘再相加”，每一列得到一个数。

小例子：x = [1, 2]，W 是 2 行 3 列

```
W = [[1.0, 0.0, -1.0],
     [0.5, 2.0,  1.0]]
```

- 第 1 列：1×1.0 + 2×0.5 = 2
- 第 2 列：1×0.0 + 2×2.0 = 4
- 第 3 列：1×(−1.0) + 2×1.0 = 1

结果 [2, 4, 1]：**2 个数进去，3 个数出来**。`nn.Linear(64, 65)` 就是一张 64 行 65 列的表，64 个数进去，65 个数出来。表里的数就是模型要学的**参数**。

```
x = torch.tensor([1.0, 2.0])
W = torch.tensor([[1.0, 0.0, -1.0], [0.5, 2.0, 1.0]])
print(x @ W)    # tensor([2., 4., 1.])   @ 就是矩阵乘法
```

张量形状 (B, T, C)：代码里一次处理很多数据，所以数据是“一摞表格”：

- **B**（batch）= 一次同时处理几段文本，这里 32
- **T**（time）= 每段多少个字，这里 64

- **C**（channel）= 每个字用几个数描述，这里 64

`(32, 64, 64) @ (64, 65)` → `(32, 64, 65)`：每段、每个位置都得到 65 个分数。读代码时，**盯住形状**比盯住数字有用得多。

本层结论：分数 = 这个位置的 64 维向量 × 一张可学习的表格。

第 3 层：字怎么变成数字？

计算机只会算数。第一步：给 65 个字符编号（`stoi`），比如 'a' → 39。但编号本身没有意义（39 并不比 38 “大”）。所以每个编号再去**查表**，换成一个 64 维向量。

回溯框 3：embedding（查表）

`nn.Embedding(65, 64)` 是一张 65 行 64 列的表，第 i 行就是第 i 个字符的向量。输入编号 i，取出第 i 行，就这么简单。

```
emb = torch.nn.Embedding(5, 3)          # 5 个字符，每个 3 维（演示用）
print(emb(torch.tensor([2, 0])))        # 取第 2 行和第 0 行
# [[ 1.5621,  1.1302, -1.4599],
#  [ 2.2092,  0.1496,  0.1491]]
```

和矩阵乘法的关系：把编号 2 写成 one-hot 向量 [0, 0, 1, 0, 0]（只有第 2 位是 1），乘上这张表，结果正好就是第 2 行。所以查表 = 一种特别省事的矩阵乘法，表里的数一样是被训练出来的。

位置编码：同一个字放在第 1 位和第 50 位，意思可能不一样。所以还有第二张表 `pos_emb`（64 行，每个位置一行），把“字向量 + 位置向量”相加，作为这个位置的起点：

```
x = self.tok_emb(idx) + self.pos_emb(torch.arange(T))   # (B, T, 64)
```

本层结论：编号 → 查表得到字向量 → 加上位置向量。到这里，每个位置有了一个 64 维向量，但它**只知道自己是什么字**，还不知道上下文。

第 4 层：一个字怎么“看到”前面的字？

要猜 “to be or not to b” 的下一个字，光看最后的 b 不够，得看前面。**注意力**就是让每个位置去前面“找相关的字，把它们的信息按相关程度混进来”。

它靠两个工具：**点积**判断“相关不相关”，**加权平均**完成“混进来”。

回溯框 4：点积 = 相似度；加权平均 = 按比例混合

点积：两个向量对应位置相乘，再全部加起来，得到一个数。

- a = [1, 2]，b = [2, 4]（方向相同）：1×2 + 2×4 = **10**，很大
- a = [1, 2]，c = [−2, 1]（方向垂直）：1×(−2) + 2×1 = **0**

方向越接近，点积越大。所以点积可以当“相似度打分”。

加权平均：权重加起来等于 1，按比例混合。权重 [0.7, 0.2, 0.1]，三个值 [10, 20, 30]：

0.7×10 + 0.2×20 + 0.1×30 = 7 + 4 + 3 = **14**

结果最像权重最大的那个（10），但也带了一点别的。

```
a, b, c = torch.tensor([1., 2.]), torch.tensor([2., 4.]), torch.tensor([-2., 1.])
print(a @ b, a @ c)                                          # 10. 0.
print(torch.tensor([0.7, 0.2, 0.1]) @ torch.tensor([10., 20., 30.]))   # 14.
```

注意：第 1 层的 softmax 正好能把一组打分变成“加起来等于 1 的权重”。三样工具在下一层拼起来。

本层结论：注意力 = 用点积给前面每个字打相关分 → softmax 变成权重 → 按权重把它们的信息加权平均进来。

第 5 层：Q/K/V 和因果掩码是什么？

直接拿两个字的向量做点积不够灵活。注意力把每个位置的向量 x 通过三张不同的表（三个 `nn.Linear`）变成三个向量：

名字	比喻	作用
Q（query，查询）	我在找什么	当前位置发出的“问题”
K（key，键）	我能提供什么	每个位置挂出的“标签”
V（value，值）	真被拿走的内容	被选中后贡献出去的信息

当前位置的 Q 和每个位置的 K 做点积 → 得到相关分 → softmax 变权重 → 对 V 做加权平均。为什么要分开 K 和 V？因为“用来被找到的特征”和“被找到后要交出的信息”不一定一样，就像图书馆的索引卡和书本身。

对应 `gpt.py` 的 `Head.forward`，一共 5 行：

```
k, q, v = self.key(x), self.query(x), self.value(x)           # (B,T,16) 各一份
wei = q @ k.transpose(-2, -1) * k.shape[-1] ** -0.5             # (B,T,T) 打分
wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))    # 因果掩码
wei = F.softmax(wei, dim=-1)                                    # 变成权重
return wei @ v                                                  # 加权平均
```

`q @ k.transpose(-2, -1)` 是一次把“每个位置 × 每个位置”的点积全算出来，得到一张 T×T 的打分表（transpose 是把 K 转个方向，好让矩阵乘法对上形状）。

回溯框 5a：因果掩码（不许偷看后面）

训练时一段 64 个字是同时喂进去的，每个位置都要猜**它的下一个字**。如果第 3 个位置能看到第 4 个字，那它直接“抄答案”就行，什么也学不到；而真正生成时，后面的字根本还不存在。

做法：用一个**下三角矩阵**（对角线及以下是 1，以上是 0）当“允许看”的名单，把不允许的位置打分改成 **−∞**。因为 e^(−∞) = 0，softmax 之后那些位置的权重就变成 0。

```
tril = torch.tril(torch.ones(4, 4))
s = torch.zeros(4, 4).masked_fill(tril == 0, float('-inf'))
print(F.softmax(s, dim=-1))
# [[1.00, 0.00, 0.00, 0.00],    第 1 个字只能看自己
#  [0.50, 0.50, 0.00, 0.00],    第 2 个字看前 2 个
#  [0.33, 0.33, 0.33, 0.00],
#  [0.25, 0.25, 0.25, 0.25]]    第 4 个字看全部 4 个
```

为什么用 −∞ 而不是直接把权重写 0？因为改完还要过 softmax 重新归一化，只有 −∞ 能保证被屏蔽的位置分到 0，其余位置的权重仍加起来等于 1。

回溯框 5b：为什么要乘 `k.shape[-1] ** -0.5`（除以 √d）

向量越长（维度 d 越大），点积是越多项相加，数值波动就越大。实测随机向量的点积波动（标准差）：

d	点积波动	除以 √d 后
4	1.99	0.99
64	8.01	1.00
1024	32.6	1.02

分数太大时，softmax 会变得“非黑即白”：[30, 20, 10] 过 softmax 是 [0.99995, 0.00005, 0.000000002]，几乎只看一个字；[3, 2, 1] 则是 [0.67, 0.24, 0.09]，还能兼顾几个。前者训练时几乎学不动。除以 √d 把波动拉回 1 左右。

多头注意力：一个头只能学一种“找法”。`MultiHeadAttention` 同时开 4 个头（每个 16 维），有的可能专找前一个字母，有的专找同一个词里的字，最后把 4 个头的结果拼起来（4×16 = 64），再过一张表 `proj` 混合。

本层结论：Q 问、K 被匹配、V 被拿走；掩码保证只看过去；除以 √d 保证 softmax 不会太极端。

第 6 层：一个 Transformer 块里还有什么？

`Block.forward` 只有两行：

```
x = x + self.sa(self.ln1(x))      # 注意力：和别的位置交流
x = x + self.ffwd(self.ln2(x))    # 前馈网络：每个位置自己消化
```

前馈网络（FeedForward）：注意力负责“收集信息”，前馈网络负责“对收集来的东西思考加工”。它是两张表夹一个 ReLU：64 → 256 → 64。

回溯框 6a：ReLU（非线性）

ReLU(x) = max(0, x)：负数变 0，正数不变。`F.relu([-2, 0, 3])` = [0, 0, 3]。

为什么需要它？两次矩阵乘法连在一起，效果等于一次矩阵乘法（表 A 乘表 B 还是一张表）。中间夹一个“折一下”的函数，网络才能表示弯曲的、复杂的规律。

回溯框 6b：残差连接 `x = x + ...`

每一层不是把 x **替换**成新结果，而是在 x 上**加一点修改**。好处有两个：

1. **信息不丢**：原始信息一直沿着“x”这条主干往后传，每层只负责补充。
2. **好训练**：第 8 层会讲，训练时有一个“信号”要从最后一层往回传到第一层。加号让这个信号有一条直通的高速路，层数多了也不会在半路衰减没了。

比喻：不是每层重写一篇文章，而是每层在旁边加批注。

回溯框 6c：均值、方差与 LayerNorm

均值：平均数。[2, 4, 6, 8] 的均值 = 20 ÷ 4 = **5**。

方差：每个数离均值多远，平方后取平均。离均值的差是 [−3, −1, 1, 3]，平方 [9, 1, 1, 9]，平均 = 20 ÷ 4 = **5**。方差的平方根（标准差）≈ 2.236，表示“数值的散开程度”。

LayerNorm：对每个位置的 64 个数，先减均值、再除以标准差，让它们变成“均值 0、散开程度 1”：

(2−5)/2.236 = −1.34，(4−5)/2.236 = −0.45，(6−5)/2.236 = 0.45，(8−5)/2.236 = 1.34

```
h = torch.tensor([2., 4., 6., 8.])
print(F.layer_norm(h, (4,)))    # tensor([-1.3416, -0.4472, 0.4472, 1.3416])
```

（真实的 `nn.LayerNorm` 还会再乘一组、加一组可学习的数，让模型自己决定最终的尺度。）

> **它解决什么**：经过很多层的累加，向量里的数可能越来越大或越来越小，后面的层就很难处理。LayerNorm 在每次进入注意力/前馈网络前把尺度“校准”一下，训练更稳定。`gpt.py` 是在进入子层**之前**做（Pre-LN），这是现在主流模型的做法。

整体结构：

```
编号 → 字向量 + 位置向量
     → [Block] × 4       （每块：LN → 注意力 → 加回；LN → 前馈 → 加回）
     → LayerNorm → lm_head → 65 个分数 → softmax → 概率
```

到这里，**模型怎么算出下一个字**已经全部拆完了。剩下的问题：那些表格里的数，是怎么被学出来的？

第 7 层：它怎么知道自己猜得好不好？

训练数据里答案是已知的：“to be or not to b” 后面就是 “e”。如果模型给正确答案的概率高，就算猜得好。衡量方法叫**交叉熵损失**（loss）。

> **回溯框 7：log 与交叉熵**
>
> **log（自然对数）**：e^x 的反操作。只需记住它的形状：
>
> - log(1) = 0
> - 数越接近 0，log 越是很大的负数
> - log 是单调的：概率越大，log 越大
>
> **交叉熵 = −log(给正确答案的概率)**
>
> 接着第 1 层的例子，概率 [0.659, 0.242, 0.099]：
>
如果正确答案是	给它的概率	loss = −log(p)
第 0 个	0.659	0.417
第 1 个	0.242	1.417
第 2 个	0.099	2.317
>
> 猜对且很有把握 → loss 接近 0；给正确答案的概率越小 → loss 越大。
>
```
F.cross_entropy(torch.tensor([[2.0, 1.0, 0.1]]), torch.tensor([0]))   # 0.4170
```
>
> **一个好用的参照**：完全瞎猜时，每个字概率都是 1/65，loss = −log(1/65) = **4.17**。第 0 层训练前的 loss 是 4.34，正好接近瞎猜；训练后降到 1.6 左右，相当于平均给正确答案约 e^(−1.6) ≈ 0.2 的概率，比瞎猜的 0.015 高了十几倍。

`gpt.py` 里同时打印 train loss 和 val loss。val（验证集）是训练时**从没见过**的 10% 文本，它才代表真实水平。

本层结论：loss 是一个数，越小说明模型给正确答案的概率越高。训练的目标就是把它往下压。

第 8 层：它怎么越学越好?

模型里有 21 万个参数（所有表格里的数）。问题变成：**每个参数该往大调还是往小调，调多少，才能让 loss 变小?**

回溯框 8a：导数 = 斜率 = “推一下，变多少”

参数 w 增加一点点，loss 跟着变化多少，这个比例就是 loss 对 w 的**导数**（也叫梯度）。

- 导数是**正**的：w 变大 → loss 变大，所以应该把 w **调小**
- 导数是**负**的：w 变大 → loss 变小，所以应该把 w **调大**
- 导数的**绝对值大**：这个参数影响大，多调一点

一句话：**沿着导数的反方向走**，loss 就会下降。

回溯框 8b：梯度下降

更新规则：**w ← w − 学习率 × 导数**

玩具例子：loss = $(w-2)^2$，最低点在 w = 2。从 w = 5 出发，学习率 0.1：

步	w	loss	导数
0	5.000	9.000	6.000
1	4.400	5.760	4.800
2	3.920	3.686	3.840
3	3.536	2.359	3.072
4	3.229	1.510	2.458

每一步都朝 2 靠近，loss 一路下降。第 0 步：5 − 0.1 × 6 = 4.4。

```
w = torch.tensor(5.0, requires_grad=True)
for i in range(5):
    loss = (w - 2) ** 2
    loss.backward()                 # PyTorch 自动算出导数，放在 w.grad
    with torch.no_grad(): w -= 0.1 * w.grad
    w.grad = None                   # 清零，否则下一轮会累加
```

`gpt.py` 训练循环做的是同一件事，只是对 21 万个参数同时做，且用了更聪明的 AdamW（会根据历史自动调整每个参数的步长）：

```
_, loss = model(*get_batch('train'))   # 前向：算 loss
opt.zero_grad(set_to_none=True)        # 清零旧梯度
loss.backward()                        # 反向：算所有参数的导数
opt.step()                             # 更新所有参数
```

回溯框 8c：反向传播与链式法则（只讲直觉）

loss 和第一层的参数之间隔了几十步计算。怎么知道第一层的参数推一下，loss 变多少？

链式法则：汽车是自行车的 2 倍快，自行车是步行的 4 倍快，汽车就是步行的 8 倍快。每一段的“放大倍数”乘起来，就是总的放大倍数。

反向传播：从 loss 出发往回走，每经过一步运算，就乘上这一步的局部放大倍数，一路传到每个参数。`loss.backward()` 就是 PyTorch 帮你自动做完这一整趟。

你只需要知道它在做什么，不需要自己算。手算版本（计算图、micrograd）都放在深挖章节。

本层结论：前向算 loss → backward 算出每个参数的导数 → 每个参数往反方向挪一小步 → 重复几千次。

第 9 层：训练为什么会出问题？

只记结论，排查时对照用：

现象	常见原因	怎么办
loss 不降反升，或变成 nan	学习率太大，一步跨过了最低点	学习率调小（如 1e-3 → 3e-4）
loss 降得非常慢	学习率太小	适当调大
train loss 一直降，val loss 开始升	**过拟合**：在背训练集，不是学规律	加 dropout、用更多数据、早停
第 0 步 loss 远大于 4.17	初始化不好，一开始就“盲目自信”	检查初始化；第 0 步 loss 应接近瞎猜的值
深层网络学不动	信号在层间放大或衰减	残差连接 + LayerNorm（`gpt.py` 已有）

本次 CPU 运行：train **1.6025**，val **1.7812**。val 略高于 train 是正常的，两者没有越拉越开，说明没有明显过拟合。

本批完成标准

1. **跑通**：运行 `python gpt.py cpu 3000`（或让我帮你跑），看到 loss 从约 4.3 降到 1.x，并生成一段像剧本的文字。
2. **用自己的话回答**（几句话就行，不需要公式）：
    - Q、K、V 各自负责什么？
    - 为什么需要因果掩码？如果去掉会怎样？
    - 残差连接和 LayerNorm 各解决什么问题？
3. **可选小实验**（任选一个，说说你看到了什么）：
    - 把 `lr` 改成 `1e-1`，loss 会怎样？
    - 把 `n_layer` 从 4 改成 1，最终 val loss 变好还是变差？

完成后把你的回答发给我，我点评后发第 2 批：**前沿架构**（RoPE、RMSNorm、SwiGLU、GQA、MoE……），同样从“现代模型和这个小 GPT 有什么不同”开始往回拆。

深挖章节（以后再看）：手算导数与计算图 · 手写 micrograd · 逐行手敲 GPT · 各公式完整推导。素材已保存。